

JAVASCRIPT

Variables, Data Types & Basic User Input

VARIABLES

var / let / const

DATA TYPES

primitive + reference

INPUT

prompt + Number

OUTPUT

console + typeof

Table of Contents

A page-by-page learning path. Every major concept begins on a fresh page.

PAGE	SECTION	PAGE	SECTION
03	Introduction to JavaScript	20	Array
04	First Program - console.log()	21	Object
05	Variables in JavaScript	22	prompt()
06	Declaration	23	Number() Conversion
07	Initialization	24	Add Two Numbers
08	Declaration + Initialization	25	typeof Operator
09	var, let and const	26	Common Mistakes
10	Re-declaration	27	Comparison Tables
11	Re-assignment	28	Quick Revision
12	Data Types Overview	29	Practice Questions - 1 to 15
13	String	30	Practice Questions - 16 to 25
14	Number	31	Answer Key - 1 to 15
15	Boolean	32	Answer Key - 16 to 25
16	Undefined	33	Mini Project - Simple Calculator
17	Null	34	Session Summary
18	BigInt	35	Clean Session Code
19	Non-Primitive / Reference Types		

Introduction to JavaScript

The language that adds behavior and logic to web pages.

What is JavaScript?

JavaScript is a programming language widely used to make web pages interactive and dynamic. In this session, we focus on basic language building blocks: variables, data types, input, output, conversion and simple arithmetic.

Where JavaScript is commonly used

- Browser-based web applications and interactive interfaces
- Server-side applications through JavaScript runtimes such as Node.js
- Small scripts, automation and many other programming environments

Session goal

By the end of this session, you should be able to create variables, identify common data types, take simple browser input, convert text input into numbers and display results.

First Program - console.log()

Start with output: tell JavaScript what to display.

The simplest program in this session prints text to the console.

Basic output

```
console.log("Hello");
```

What happens here?

`console.log()` sends a value to the console. The text inside quotes is a string. In a browser, the result can be viewed in the Developer Tools console.

Expected output

Hello

Remember

A semicolon is commonly used to terminate a JavaScript statement. JavaScript can often insert semicolons automatically, but consistent semicolon use improves readability.

Variables in JavaScript

A variable gives a name to data that your program needs to work with.

JavaScript provides three variable declaration keywords: `var`, `let` and `const`. Their rules differ, especially for redeclaration and reassignment.

Three declarations

```
var a;  
let b;  
const c = 10;
```

Core idea

Think of a variable as a named place your program can use to access a value. The keyword you choose determines what changes are permitted later.

Keyword	Beginner meaning
var	Older declaration style; can be redeclared and reassigned.
let	Use when the variable value may be reassigned.
const	Use when the binding should not be reassigned; initialize it immediately.

Declaration

Create the variable name before assigning a value.

Declaration

```
var a;  
let b;
```

Declaration means introducing a variable to the program. At this point, no explicit value has been assigned.

Important

`var a;` and `let b;` are valid declarations. `const` is different: a `const` declaration must include an initializer.

const rule

```
const c; // SyntaxError  
  
const c = 10; // Valid
```

Initialization

Give a declared variable its first value.

Declaration followed by assignment

```
var a;  
let b;  
  
a = 10;  
b = 20;
```

Initialization is the act of giving a variable an initial value. For `var` and `let`, declaration and value assignment may be separated.

Mental model

Declaration creates the named variable. Initialization gives it its first useful value.

Flow

Simple flow

```
let b;    ->    b = 20;
```

Declaration + Initialization

Both steps can happen in one statement.

Combined form

```
var x = 70;  
let y = 20;  
const z = 43;
```

In these statements, the variable is declared and given its first value immediately. This is the most common way you will write variables in everyday code.

Read this line

`let y = 20;` -> `let` is the keyword, `y` is the variable name, and `20` is the initial value.

Structure

```
let y = 20;  
|   |   |  
|   |   +-- value  
|   +----- variable name  
+----- keyword
```

var, let and const

The three declaration keywords have different rules.

Feature	var	let	const
Declaration without initialization	Yes	Yes	No
Redeclaration in same scope	Yes	No	No
Reassignment	Yes	Yes	No
Must initialize immediately	No	No	Yes

Practical rule

For modern JavaScript, `let` and `const` are generally preferred. Use `let` when reassignment is needed and `const` when the binding should not be reassigned.

Re-declaration

Declare the same variable name again.

Examples

```
var x = 70;  
var x = 40;  
  
let y = 20;  
// let y = 40; // Error  
  
const z = 43;  
// const z = 50; // Error
```

Definition

Redeclaration means creating another declaration using the same variable name in the same scope. `var` permits this; `let` and `const` do not.

Exam point

Do not confuse redeclaration with reassignment. They are different operations.

Re-assignment

Change the value of an existing variable.

Reassignment

```
var x = 40;  
let y = 20;  
const z = 43;  
  
x = 66;  
y = 44;  
// z = 50; // Error
```

Concept	Meaning	Example
Redeclaration	Declare the same variable name again.	<code>`var x; var x;`</code>
Reassignment	Change the current value of an existing variable.	<code>`x = 66;`</code>

Key rule

``var`` and ``let`` can be reassigned. A ``const`` binding cannot be reassigned after initialization.

Data Types Overview

JavaScript values can be classified as primitive values or objects/reference types.

Category	Types in this session	Main idea
Primitive	String, Number, Boolean, Undefined, Null, BigInt	Represent individual primitive values.
Reference / object types	Array, Object	Objects can hold collections or structured data.

Hierarchy

JavaScript Data Types

```
|
+-- Primitive
|   +-- String
|   +-- Number
|   +-- Boolean
|   +-- Undefined
|   +-- Null
|   +-- BigInt
|
+-- Reference / object types
    +-- Array
    +-- Object
```

Accuracy note

For beginner study, arrays and objects are commonly discussed as reference types. Avoid reducing JavaScript memory behavior to a single slogan such as "primitive = stored directly, object = stored by reference"; the language model is more nuanced.

String

Textual data enclosed in quotes.

String examples

```
let str = "Hello";  
let name = "Roshan";  
let city = 'Mumbai';
```

A string represents text. JavaScript allows string literals using single quotes or double quotes.

Value	Type	Purpose
"Hello"	string	Text
'Mumbai'	string	Text

Remember

Quotes are part of the syntax used to create the string literal; the actual string value does not include the surrounding quote characters.

Number

The JavaScript `number` type covers integers and floating-point values.

Number examples

```
let num = 56;  
let num1 = 12.43;
```

Example	Kind	JavaScript type
56	Integer	number
12.43	Floating-point	number

Remember

You do not use separate JavaScript types called "integer" and "float" in this beginner-level model. Both examples above have the type `number`.

Boolean

A Boolean value expresses one of two logical states.

Boolean examples

```
let isReady = true;  
isReady = false;
```

Value	Beginner-friendly interpretation
true	condition is satisfied / on
false	condition is not satisfied / off

Important

Boolean is a real data type with exactly two values: `true` and `false`. The words "yes" and "no" are only useful analogies for learning.

Undefined

A variable can exist without having an assigned value.

Undefined example

```
let value;  
  
console.log(value);  
console.log(typeof value);
```

A declared `let` or `var` variable that has not been assigned a value evaluates to `undefined`. The `typeof` operator reports its type as `"undefined"`.

Expected output

undefined "undefined"

Null

An intentional representation of "no value".

Null example

```
let nullVal = null;  
console.log(nullVal);  
console.log(typeof nullVal);
```

Value	Typical meaning
undefined	No value has been assigned in the usual declaration case.
null	The programmer intentionally represents an absent or empty value.

JavaScript quirk

``typeof null`` returns ```object```. This is a long-standing historical behavior in JavaScript; it does not mean that ``null`` is actually an object.

BigInt

Represent integers that are too large for exact ordinary `number` handling.

BigInt examples

```
let bigNum = 2342323534523453464575686785745n;  
  
let safeBoundary = 9007199254740991n;
```

BigInt is a separate numeric type for integers beyond the exact integer range of ordinary JavaScript `Number` values. The trailing `n` identifies a BigInt literal.

Remember

BigInt is for integers. Do not treat it as just a longer decimal `number`; it is a separate type with its own arithmetic rules.

Non-Primitive / Reference Types

Arrays and objects are the structured values used in this session.

An array stores an ordered collection. An object stores related information using properties (key-value pairs). In beginner-level discussions, both are commonly described as reference types because variables refer to object values.

Focus for Session 1

You only need two examples today: arrays and objects. Later JavaScript sessions can build on these concepts with methods, nested structures and more advanced object behavior.

Array

Store multiple ordered values in one collection.

Array example

```
const arr = [1, 2, 3, 4, 5, 6, 6, 7];  
  
console.log(arr[0]); // 1  
console.log(arr[3]); // 4
```

Index	0	1	2	3	4	5	6	7
Value	1	2	3	4	5	6	6	7

Remember

Array indexing starts at `0`, not `1`. Duplicate values are allowed.

Object

Represent related information using properties.

Object example

```
const obj = {  
  id: 4,  
  key: 78  
};  
  
console.log(obj.id);  
console.log(obj["id"]);
```

Part	Meaning
id	Property / key
4	Value stored for `id`
obj.id	Dot notation
obj["id"]	Bracket notation

Pattern

Object data is written as `key: value`. This key-value structure makes objects useful for representing records and grouped information.

prompt()

Collect simple user input in a browser environment.

User Input

```
let s = prompt("Enter Num1: ");  
let g = prompt("Enter Num2: ");
```

In a browser, `prompt()` displays a dialog box and returns the user input as text (or `null` if the dialog is cancelled). For this session, the key point is that normal input arrives as a string.

Important

If the user enters `10`, the returned value is the string `"10"`, not the numeric value `10`. Convert it before arithmetic when you want numeric behavior.

Number() Conversion

Convert numeric text into a JavaScript number.

Conversion

```
let s = Number(prompt("Enter Num1: "));  
let g = Number(prompt("Enter Num2: "));
```

Conversion flow

```
"10"  
|  
+-- Number("10")  
   |  
   +-- 10
```

Expression	Result
"10" + "20"	"1020" (string concatenation)
Number("10") + Number("20")	30 (numeric addition)

Remember

`Number()` asks JavaScript to convert a value to the `number` type. Invalid numeric text can produce `NaN`, but that edge case is outside the main task in this session.

Add Two Numbers

A complete Session 1 program using input, conversion, arithmetic and type checking.

Complete program

```
let s = Number(prompt("Enter Num1: "));
let g = Number(prompt("Enter Num2: "));

console.log(
  "Addition:",
  s + g,
  "\nType of Number:",
  typeof (s + g)
);
```

Program flow

Execution flow

```
User input -> prompt() -> string
      |
      +-> Number() -> number
            |
            +-> s + g -> result
                  |
                  +-> typeof -> data type
```

Sample execution

Enter Num1: 10 Enter Num2: 20 Addition: 30 Type of Number: "number"

typeof Operator

Check the JavaScript type reported for a value.

Common examples

```
typeof "Hello";    // "string"
typeof 56;         // "number"
typeof true;       // "boolean"
typeof undefined;  // "undefined"
typeof 123n;       // "bigint"
typeof [];         // "object"
typeof {};         // "object"
typeof null;       // "object"
```

Value	typeof result
"Hello"	"string"
56	"number"
true	"boolean"
undefined	"undefined"
123n	"bigint"
[] / {}	"object"
null	"object" (historical quirk)

Common Beginner Mistakes

Use these warning boxes as a last-minute checklist.

1. `const` without initialization

`const x;` is invalid. Initialize it when declared.

2. Reassigning a `const`

`const x = 10; x = 20;` is an error.

3. Assuming `prompt()` returns a number

User input is text, so use `Number()` when numeric input is required.

4. Forgetting string concatenation

`"10" + "20"` gives `"1020"`, not `30`.

5. Confusing redeclaration with reassignment

Redeclaration creates another declaration; reassignment changes an existing value.

6. Starting array indexing at 1

The first array element is at index `0`.

7. Calling `null` an object

`typeof null` reports `"object"`, but `null` represents an intentional absence of value.

Comparison Tables

High-speed reference for revision.

Variables

Feature	var	let	const
Declare without value	Yes	Yes	No
Redeclare in same scope	Yes	No	No
Reassign	Yes	Yes	No
Initialize immediately	No	No	Yes

Data types

Type	Example	Meaning
String	"Hello"	Text
Number	56 / 12.43	Numeric values
Boolean	true	Logical state
Undefined	undefined	No assigned value in the usual declaration case
Null	null	Intentional absence
BigInt	123n	Large integer values
Array	[1,2,3]	Ordered collection
Object	{id:4}	Key-value data

Quick Revision Sheet

One page to scan before a quiz, viva or practical exam.

Variables

``var`` -> redeclaration + reassignment allowed ``let`` -> reassignment allowed, same-scope redeclaration not allowed
``const`` -> initialization required; reassignment not allowed

Data Types

Primitive in this session -> String, Number, Boolean, Undefined, Null, BigInt Reference / object types -> Array, Object

Input and conversion

``prompt()`` -> collects browser input as text ``Number()`` -> converts a value to number when conversion is valid ``typeof`` -> reports the type category as a string

Three distinctions to remember

Declaration != Initialization Redeclaration != Reassignment String input != Numeric value

Practice Questions - 1 to 15

Answer without looking at the notes first.

1. Write a statement that prints ``Hello JavaScript`` using ``console.log()``.
2. Declare one variable using ``var`` and one using ``let`` without assigning a value immediately.
3. Write a valid ``const`` declaration with the value ``25``.
4. Explain the difference between declaration and initialization in one or two sentences.
5. Can a ``var`` variable be redeclared in the same scope? Write a small example.
6. Can a ``let`` variable be reassigned? Write a small example.
7. Why is ``const x;`` invalid?
8. Create a string variable named ``course`` with the value ``JavaScript``.
9. Create one integer-like ``number`` value and one decimal ``number`` value.
10. Create a Boolean variable and change its value from ``true`` to ``false``.
11. What value does a declared ``let`` variable normally have before an explicit value is assigned?
12. Create an array containing five numbers and print the first element.
13. Create an object with properties ``id`` and ``name``, then access the ``name`` property using dot notation.
14. Write one example of redeclaration and one example of reassignment, and label each clearly.
15. Create a BigInt literal and state what the trailing ``n`` means.

Practice Questions - 16 to 25

These 10 questions are the additional concept-based questions.

16. Explain the difference between redeclaration and reassignment.
17. What is the difference between `undefined` and `null`?
18. Why does JavaScript use `Number()` around `prompt()` for arithmetic examples?
19. Predict the result of `"10" + "20"`.
20. Predict the result of `Number("10") + Number("20")`.
21. What does `typeof true` return?
22. What does `typeof 123n` return?
23. What does `typeof null` return, and why should you treat that result as a JavaScript quirk?
24. An array is `const marks = [45, 60, 72];`. Which index contains `72`?
25. Write a program that asks for two numbers and prints their sum and the type of the result.

Question count

The booklet contains 25 total practice questions: 15 core questions plus 10 additional concept-based questions.

Answer Key - 1 to 15

Concise answers for self-checking.

1. ``console.log("Hello JavaScript");``
2. ``var a;`` and ``let b;``
3. ``const value = 25;``
4. Declaration introduces the variable; initialization gives it its initial value.
5. Yes. Example: ``var x = 1; var x = 2;``
6. Yes. Example: ``let x = 1; x = 2;``
7. ``const`` requires an initializer at declaration time.
8. ``let course = "JavaScript";``
9. Example: ``let a = 10; let b = 12.5;``
10. ``let ready = true; ready = false;``
11. ``undefined``.
12. Example: ``const nums = [10,20,30,40,50]; console.log(nums[0]);``
13. Example: ``const user = { id: 1, name: "Asha" }; console.log(user.name);``
14. Redeclaration: ``var x = 1; var x = 2;`` Reassignment: ``x = 3;``
15. Example: ``const big = 12345678901234567890n;`` The ``n`` suffix identifies a BigInt literal.

Answer Key - 16 to 25

Concise answers for the remaining questions.

- 16. Redefinition creates another declaration with the same name; reassignment changes the value of an existing variable.
- 17. ``undefined`` commonly means no value has been assigned; ``null`` is an intentional absence value.
- 18. Because ``prompt()`` returns text, and numeric arithmetic requires numeric values.
- 19. ``"1020"``.
- 20. ``30``.
- 21. ``"boolean"``.
- 22. ``"bigint"``.
- 23. ``typeof null`` is `"object"`; this is a historical JavaScript behavior, not evidence that null is an object.
- 24. Index ``2``.
- 25. Example: ``let a = Number(prompt("Enter Num1: "));`
`let b = Number(prompt("Enter Num2: "));`
`console.log(a + b, typeof (a + b));``

Mini Project - Simple Calculator

Combine Session 1 concepts into one small browser exercise.

Task

Create a program that asks the user for two numbers and displays their addition, subtraction, multiplication and division results. Also display the type of at least one numeric result.

Requirements

- Use `prompt()` for both inputs.
- Use `Number()` to convert both values.
- Use `console.log()` for output.
- Use arithmetic operators `+`, `-`, `*`, `/`.
- Use `typeof` on a numeric result.

Suggested flow

Input -> Convert -> Calculate -> Display -> Check type

Session Summary

What you should be able to explain and code after Session 1.

Core topics

- `console.log()` for output
- Variables: `var`, `let`, `const`
- Declaration and initialization
- Redeclaration and reassignment
- Primitive data types
- Arrays and objects
- `prompt()` for browser input
- `Number()` for numeric conversion
- `typeof` for type checking

What you should be able to do now

Declare and initialize variables; choose between `let` and `const` at a beginner level; explain redeclaration vs reassignment; identify common data types; build simple arrays and objects; accept numeric input; perform arithmetic; and check a result type.

Next learning direction

The next JavaScript session can build naturally on these basics with operators, conditions, or other topics covered by your course. Do not skip the fundamentals: variable rules and type behavior will appear repeatedly later.

Clean Session Code

A normalized version of the original code used in this booklet.

Session 1 code

```
console.log("Hello");

// Variables
var a;
let b;
const c = 10;

a = 10;
b = 20;

var x = 70;
let y = 20;
const z = 43;

var x = 40; // redeclaration
x = 66;     // reassignment
y = 44;     // reassignment

// Data types
let str = "Hello";
let num = 56;
let num1 = 12.43;

let isReady = true;
isReady = false;

let nullVal = null;
let bigNum = 2342323534523453464575686785745n;

// Array
const arr = [1, 2, 3, 4, 5, 6, 6, 7];

// Object
const obj = {
  id: 4,
  key: 78
};

// User input
let s = Number(prompt("Enter Num1: "));
let g = Number(prompt("Enter Num2: "));

console.log(
  "Addition:",
  s + g,
  "\nType of Number:",
  typeof (s + g)
);
```

Note

The appendix keeps the original session scope while applying consistent formatting and correcting the wording around `const`, `null` and reference types in the explanatory pages.